

Replacing a Principal Object

09/15/2021

Applications that provide authentication services must be able to replace the **Principal** object ([IPrincipal](#)) for a given thread. Furthermore, the security system must help protect the ability to replace **Principal** objects because a maliciously attached, incorrect **Principal** compromises the security of your application by claiming an untrue identity or role. Therefore, applications that require the ability to replace **Principal** objects must be granted the [System.Security.Permissions.SecurityPermission](#) object for principal control. (Note that this permission is not required for performing role-based security checks or for creating **Principal** objects.)

The current **Principal** object can be replaced by performing the following tasks:

1. Create the replacement **Principal** object and associated **Identity** object.
2. Attach the new **Principal** object to the call context.

Example

The following example shows how to create a generic principal object and use it to set the principal of a thread.

C#

```
using System;
using System.Threading;
using System.Security.Permissions;
using System.Security.Principal;

class SecurityPrincipalDemo
{
    public static void Main()
    {
        // Retrieve a GenericPrincipal that is based on the current user's
        // WindowsIdentity.
        GenericPrincipal genericPrincipal = GetGenericPrincipal();

        // Retrieve the generic identity of the GenericPrincipal object.
        GenericIdentity principalIdentity =
            (GenericIdentity)genericPrincipal.Identity;

        // Display the identity name and authentication type.
```

```
if (principalIdentity.IsAuthenticated)
{
    Console.WriteLine(principalIdentity.Name);
    Console.WriteLine("Type:" + principalIdentity.AuthenticationType);
}

// Verify that the generic principal has been assigned the
// NetworkUser role.
if (genericPrincipal.IsInRole("NetworkUser"))
{
    Console.WriteLine("User belongs to the NetworkUser role.");
}

Thread.CurrentPrincipal = genericPrincipal;
}

// Create a generic principal based on values from the current
// WindowsIdentity.
private static GenericPrincipal GetGenericPrincipal()
{
    // Use values from the current WindowsIdentity to construct
    // a set of GenericPrincipal roles.
    WindowsIdentity windowsIdentity = WindowsIdentity.GetCurrent();
    string[] roles = new string[10];
    if (windowsIdentity.IsAuthenticated)
    {
        // Add custom NetworkUser role.
        roles[0] = "NetworkUser";
    }

    if (windowsIdentity.IsGuest)
    {
        // Add custom GuestUser role.
        roles[1] = "GuestUser";
    }

    if (windowsIdentity.IsSystem)
    {
        // Add custom SystemUser role.
        roles[2] = "SystemUser";
    }

    // Construct a GenericIdentity object based on the current Windows
    // identity name and authentication type.
    string authenticationType = windowsIdentity.AuthenticationType!;
    string userName = windowsIdentity.Name;
    GenericIdentity genericIdentity =
        new GenericIdentity(userName, authenticationType);

    // Construct a GenericPrincipal object based on the generic identity
    // and custom roles for the user.
```

```
GenericPrincipal genericPrincipal =  
    new GenericPrincipal(genericIdentity, roles);  
  
    return genericPrincipal;  
}  
}
```

See also

- [System.Security.Permissions.SecurityPermission](#)
- [Principal and Identity Objects](#)
- [ASP.NET Core Security](#)